
Solving the Lorenz-1960 ODE System Using Optimal ANN Architectures

By

Ikramul Hasan Moral (0112230489)

Md. Abu Bakar (0112230200)

Samiur Rahman Omlan (0112230195)

Fariha Islam (0112230431)

Md. Touhidul Islam (0112230435)

Submitted in partial fulfilment of the requirements
of the degree of Bachelor of Science in Computer Science and Engineering

April 10, 2026



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
UNITED INTERNATIONAL UNIVERSITY

Abstract

Nonlinear coupled ordinary differential equations (ODEs) occur often in physical modeling, and in most cases, their analytical solutions cannot be obtained. The orthodox method of solving such problems is numerical methods like the Runge-Kutta family and these are not free in computation and mesh dependency. That system is the Lorenz-1960 ODE system, a simplified meteorological model, which serves as a test case in this project, and poses a straightforward question: is it possible to use a plain artificial neural network (ANN), with neither physics informed loss nor operator learning configuration, to learn to approximate the solution to this coupled system? In response to this, an accurate reference solution is initially calculated using the Runge-Kutta based solver in the SciPy. There are several feedforward ANN structures that are then trained on this reference data, and the different variations are the number of hidden layers, the number of neurons within any given layer and the activation function. The models are tested using approximation error, training convergence, as well as time of computation. The purpose of this comparison is to produce the most appropriate architectural configuration of this particular ODE system and to contextualize the results with regard to the more recent literature on neural network based solvers of differential equations. The final product of this study is the possession of an empirically-based opinion regarding what plain ANNs are and are not capable of as ODE solvers.

Acknowledgements

This work would have not been possible without the input and support of many people over the last two trimesters. We would like to express my gratitude to everyone who contributed to it in some way or other.

First, we would like to thank my academic advisors, ...

Our sincere gratitude goes to ...

We are also thankful to ...

Last but not the least, We owe to our family including our parents for their unconditional love and immense emotional support.

Publication List

[Optional] The main contributions of this research are either published or accepted or in preparation in journals and conferences as mentioned in the following list:

Journal Articles

- 1.

Conference Papers

- 1.

Additional Publications

Following is the list of relevant publications published in the course of the research that is not included in the thesis:

- 1.

Table of Contents

Table of Contents	v
List of Figures	vi
List of Tables	vii
1 Introduction	1
1.1 Project Overview	1
1.2 Motivation	1
1.3 Objectives	2
1.4 Methodology	2
1.5 Project Outcome	3
1.6 Organization of the Report	4
2 Background	5
2.1 Preliminaries	5
2.1.1 Ordinary Differential Equations	5
2.1.2 The Lorenz-1960 System	5
2.1.3 Classical Numerical Methods	6
2.1.4 Artificial Neural Networks	6
2.1.5 ANN-Based ODE Solving	6
2.1.6 Physics-Informed Neural Networks	7
2.2 Literature Review	7
2.2.1 Similar Applications	7
2.2.2 Related Research	8
2.3 Gap Analysis	9
2.4 Summary	10
3 Project Design	11
3.1 Requirement Analysis	11
3.1.1 Functional and Nonfunctional Requirements	11
3.1.2 Context Diagram	11
3.1.3 Data Flow Diagram Level 1	11

3.1.4	UI Design	11
3.2	Detailed Methodology and Design	11
3.3	Project Plan	11
3.4	Task Allocation	11
3.5	Summary	11
4	Implementation and Results	12
4.1	Environment Setup	12
4.2	Testing and Evaluation	12
4.3	Results and Discussion	12
4.4	Summary	12
5	Standards and Design Constraints	13
5.1	Compliance with the Standards	13
5.1.1	Software Standards	13
5.1.2	Hardware Standards	13
5.1.3	Communication Standards	13
5.2	Design Constraints	13
5.2.1	Economic Constraint	14
5.2.2	Environmental Constraint	14
5.2.3	Ethical Constraint	14
5.2.4	Health and Safety Constraint	14
5.2.5	Social Constraint	14
5.2.6	Political Constraint	14
5.2.7	Sustainability	14
5.3	Cost Analysis	14
5.4	Complex Engineering Problem	14
5.4.1	Complex Problem Solving	14
5.4.2	Engineering Activities	14
5.5	Summary	14
6	Conclusion	16
6.1	Summary	16
6.2	Limitation	16
6.3	Future Work	16
	References	18

List of Figures

1.1	Methodology Diagram	3
-----	-------------------------------	---

List of Tables

2.1	Literature Gap Analysis	9
5.1	Mapping with complex problem solving.	14
5.2	Mapping with complex engineering activities.	15

Chapter 1

Introduction

This chapter introduces the research topic, provides an overview and motivation for the study, and outlines the main objectives and expected outcomes. It also outlines the methodology and briefly describes how the rest of the report is organized.

1.1 Project Overview

Ordinary Differential Equations (ODEs) describe the dynamic changes of quantities over time and are widely used in fields such as physics, engineering, and mathematics. In many real-world problems, particularly those involving complex or nonlinear equations, it is often not possible to obtain exact solutions. That is why methods such as Euler's method and Runge-Kutta are used to obtain approximate solutions. In this project, we examine a particular system consisting of three nonlinear equations known as the Lorenz-1960 system (i.e., one of the first meteorological models). The system is small but complex enough to make a good testing ground for various solution methods. The Lorenz-1960 system is a great candidate for evaluating how well ANNs learn and predict behavior. Classical numerical methods address these problems step by step and are typically accurate, but they tend to be slow and require substantial computing power. Generated data from numerical methods, such as Runge-Kutta, can be used to train ANNs to model the relationship between time and the variables. During training, the ANN learns to map its predictions to those of the numerical solver. Once the ANN has been trained, it can provide rapid predictions of outcomes for any time point without needing to solve the equations again. Therefore, the primary objective of this project is to solve the Lorenz-1960 system and to see how well a simple ANN can learn its behavior and which ANN setup yields optimal results.

1.2 Motivation

This project is motivated by both computational and research perspectives. Numerical methods are solid and accurate. While these numerical methods are well-suited for direct

simulation, they become less efficient when repeated evaluations are required. The subsequent solution of the equation can then use the ANN as a direct approximator. After training, the model outputs predictions and acts as a surrogate to the system. Thus, approximating the solution using an ANN is another approach. There is a growing body of literature applying neural approaches to differential equations (DE), including physics-informed neural networks (PINNs), operator-learning, and ANN-based approximators, among others. This discrepancy renders this issue both timely and appropriate for exploration. Another motivation is the educational and practical benefits of this project. This project provides foundations for fostered expertise in ODE solving, neural network modeling, comparative experimentation, and scientific analytical processes. This makes any such comparative evaluation valuable in itself, even if the results would indicate that simple ANNs are outperformed by more advanced methods - such findings will help clarify the limitations of plain ANN approaches.

1.3 Objectives

Our research paper primarily aims to look into whether a plain ANN learns the chaotic Lorenz 1960 ODE system. Some specific objectives are given below:

- As the Lorenz system is a three-coupled nonlinear, deterministic, and chaotic system, to solve this, we combine different areas such as ODE, numerical method (Runge Kutta & SciPy solver), nonlinear, and ANN model.
- To generate a high-accuracy numerical solution and validate the ANN's performance, compare ANN results with different numerical methods, a benchmark formulation was created using the traditional Runge-Kutta method.
- To evaluate several Feedforward ANN architectures, we must experiment with depth(number of hidden layers) and width(number of neurons per layer), and activation function to get the most efficient structure. This objective determines if the ANN structure can achieve the high accuracy provided by the Runge-Kutta method.
- The performance of the model will be analyzed by error metrics(mean squared error) to evaluate if a plain ANN can learn the Lorenz system.
- Compare results with the PINN-based result from [1] (PinnDE) and the survey-based results from [2].

1.4 Methodology

Our research will follow some stages, and in the beginning, we learn and understand Lorenz -1960 three equations. After that we are identifying which ANN architecture will approximate its solution and give the best outcome. In this methodology, we only cover

our FYDP 1 approach to solve the Lorez-1960 ODE system using ANN and identify the best optimal architecture for our research. Our research follow some stages:

To begin with, as the Lorez system is “chaotic”, meaning a small error can grow very fast and impact the whole system so it can be defined as an initial value problem.

And now this system will be solved using two methods: first, a validated numerical baseline using a high-accuracy Runge-Kutta solver in Python and second, a high-accuracy SciPy solver for reference comparison.

Next, to simplify supervised learning, first generate aligned reference dataset and then problem framing for learning. This resulting dataset ensures optimization against numerical baseline.

Then comes the ANN architecture design based on literature-guided parameters to ensure optimal performance. Here a family of feedforward ANN mode will be constructed based on depth, width, activation function, and optimization strategy.

After that to maintain computational efficiency on standard hardware while preserving the system’s chaotic dynamics, the data is densely sampled and partitioned into training, validation, and testing subsets. This follows the standard data-driven modeling approach explored in recent ANN surveys for differential equations [2].

And now compare against the numerical baseline. It means taking your trained ANN models and measuring how well they perform against two distinct reference points.

Lastly, comparing all ANN models against the numerical and literature based benchmarks considering factors such as structural simplicity, training stability, and reproducibility. We analyze the trade-offs among them. Which architecture will satisfy all this then it will be a recommended solution for the Lorez-1960 benchmark.

For better understanding diagram of our methodology part is given below:-

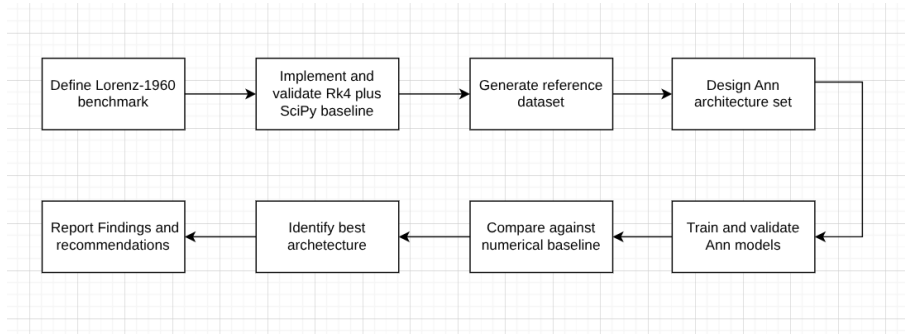


Figure 1.1: Methodology Diagram

1.5 Project Outcome

The expected outcome of the project will be the clear and the justified determination of the most appropriate architecture for the ANN in estimating the solution to the Lorez-1960 system of Ordinary Differential Equations (ODEs) within the scope of the experiment.

This project must also include a numerical reference solution to the ODEs, the establishment of ANN baselines, and a comparative analysis of the results.

The project will also contribute to the current understanding of the aptitude and limitations of ANN models in the approximation and solution of differential equations. If the ANN models are able to get a satisfactory level of accuracy in approximating the solution to the ODEs, the project will have demonstrated the capability of a simple ANN architecture to effectively act as an alternative numerical solver. This would suggest that a plain ANN, without the need for physics-informed loss terms or traditional mesh-dependent numerical methods, can on its own approximate ODE solutions with sufficient accuracy while being computationally less expensive in terms of inference time and the avoidance of iterative re-solving for each query. But if the models cannot achieve satisfactory accuracy in approximating the solution to the ODEs then the study will clearly outline the constraints of simple ANN architectures and underscore the need for advanced methods like PINNs.

The expected outcome of the project will therefore be more than just the coding and experiment itself but will also involve a research based understanding of how ANN architectural choices like depth, width, and activation function can impact solution accuracy for coupled nonlinear ODE systems.

1.6 Organization of the Report

The rest of this report is outlined as follows. Chapter 2 outlines the theoretical background and literature review on Ordinary Differential Equations (ODE) solving, artificial neural networks, and existing neural methods for differential equations. Chapter 3 discusses about the project design, requirements, methodological plan, and task assignment. Chapter 4 outlines the implementation & experiment design used in this research. Chapter 5 outlines the results and analysis on the tested ANN structures. Finally, Chapter 6 concludes this report by focusing the main results, limitations, and potential directions for the future research.

Chapter 2

Background

This chapter covers the foundational knowledge you need to understand our work. We start with the mathematical and computational preliminaries. Then we review the existing literature on ANN-based ODE solvers and similar applications. We identify the research gaps that motivate our study. Finally, we wrap up with a brief summary.

2.1 Preliminaries

This section provides the necessary background knowledge to understand the rest of the report.

2.1.1 Ordinary Differential Equations

An ODE describes how a quantity changes with respect to a single independent variable. A general first-order ODE takes the form:

$$\frac{dy}{dt} = f(t, y), \quad y(t_0) = y_0 \quad (2.1)$$

where $y(t)$ is the unknown function, t is the independent variable, and y_0 is the initial condition.

2.1.2 The Lorenz-1960 System

The Lorenz-1960 system consists of three coupled nonlinear ODEs for meteorological modeling. We target the formulation from [1] (Section 4.2):

$$\begin{aligned} \frac{dx}{dt} &= kl \left(\frac{1}{k^2 + l^2} - \frac{1}{k^2} \right) yz \\ \frac{dy}{dt} &= kl \left(\frac{1}{l^2} - \frac{1}{k^2 + l^2} \right) xz \\ \frac{dz}{dt} &= \frac{kl^2}{2} \left(\frac{1}{k^2} - \frac{1}{l^2} \right) xy \end{aligned} \quad (2.2)$$

with $k = 2$, $l = 1$, initial conditions $x(0) = 0.5$, $y(0) = 0.75$, $z(0) = 1$, and $t \in [0, 1]$. Its nonlinear, coupled dynamics make it an excellent benchmark for numerical solvers.

2.1.3 Classical Numerical Methods

Euler's method approximates the solution via a first-order Taylor expansion:

$$y_{n+1} = y_n + h \cdot f(t_n, y_n) \quad (2.3)$$

where h is the step size. It is simple but accumulates errors quickly.

The **fourth-order Runge-Kutta method (RK4)** evaluates the derivative at multiple points per step for higher accuracy:

$$y_{n+1} = y_n + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4) \quad (2.4)$$

Both methods are mesh-dependent, require re-solving for every new initial condition, and struggle with stiff equations.

2.1.4 Artificial Neural Networks

A feedforward ANN (Multilayer Perceptron) consists of an input layer, hidden layers, and an output layer. Each neuron computes:

$$a_i(\mathbf{x}) = \sigma(\mathbf{w}^{(i)} \cdot \mathbf{x} + b^{(i)}) \quad (2.5)$$

where $\mathbf{w}^{(i)}$ are weights, $b^{(i)}$ is bias, and σ is an activation function [3]. The **Universal Approximation Theorem** [3, 4] guarantees that a network with at least one hidden layer can approximate any continuous function to arbitrary accuracy. Common activation functions include:

- **Sigmoid:** $\sigma(x) = \frac{1}{1+e^{-x}}$, maps inputs to $(0, 1)$
- **Tanh:** $\sigma(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$, maps inputs to $(-1, 1)$
- **ReLU:** $\sigma(x) = \max(0, x)$, efficient but non-smooth at zero
- **GELU/Swish:** smooth ReLU approximations with better gradient flow

Training minimizes a loss function $L(\theta)$ over network parameters θ using gradient descent. Backpropagation computes gradients via the chain rule, and optimizers like Adam adapt learning rates per parameter [5, 6].

2.1.5 ANN-Based ODE Solving

Instead of stepping through the domain, ANNs learn a continuous solution function over the entire domain. The standard approach [7, 8] constructs a trial solution:

$$\psi_t(x, \mathbf{p}) = A(x) + F(x, N(x, \mathbf{p})) \quad (2.6)$$

where $A(x)$ satisfies the initial/boundary conditions exactly, and $N(x, \mathbf{p})$ is the neural network output. The loss function measures how well this trial solution satisfies the ODE:

$$L(\mathbf{p}) = \frac{1}{N} \sum_{i=1}^N \left[\frac{d\psi_t(x_i, \mathbf{p})}{dx} - f(x_i, \psi_t(x_i, \mathbf{p})) \right]^2 \quad (2.7)$$

Minimizing this loss yields a differentiable, closed-form approximate solution without requiring knowledge of the exact answer.

2.1.6 Physics-Informed Neural Networks

PINNs embed governing equations directly into the loss function [1, 4, 6]:

$$L(\theta) = L_{\text{data}} + \lambda_1 L_{\text{physics}} + \lambda_2 L_{\text{IC/BC}} \quad (2.8)$$

The physics loss uses automatic differentiation to compute derivatives of the network output. This enables training with sparse or no data, but balancing the loss terms requires careful hyperparameter tuning.

2.2 Literature Review

2.2.1 Similar Applications

Dubey et al. [5] solved coupled first-order ODEs using a 1-10-2 ANN (tanh activation), achieving a maximum error of 2.88×10^{-3} in 120 iterations. Audu et al. [9] used a similar PyTorch-based ANN, converging within 200–1500 iterations. Okereke et al. [8] bypassed backpropagation by computing weights from Gaussian Radial Basis Function regression.

Lau and Werth [3] developed the ODEN framework, showing NNs outperform RK4 for rapidly oscillating solutions with errors of $\sim 10^{-4}$. Mall and Chakraverty [7] compared arbitrary versus regression-based weight initialization on a damped vibration equation, finding regression-based weights reduce maximum deviation from 3.67% to 1.47%.

Murugesu et al. [10] combined neural networks with block numerical methods to solve higher-order ODEs directly, achieving errors near machine precision ($\sim 10^{-10}$). Pratama et al. [2] surveyed three ANN-based PDE solvers (PyDEns, NeuroDiffEq, Nangs), all using 3 hidden layers with 32 neurons and tanh activation. The Trial Approximate Solution method in NeuroDiffEq hardcodes initial conditions into the ansatz, making it a pure-ANN approach relevant to our work.

2.2.2 Related Research

Physics-Informed Approaches

Matthews and Bihlo [1] developed PinnDE, solving the Lorenz-1960 system with a DeepONet (4 hidden layers, 40 nodes, tanh, Adam optimizer). Their DeepONet learns the solution operator $G(u_0)(t, x)$ mapping initial conditions to solutions over $[0, 1]$. This is the closest baseline to our target problem, but it relies on physics-informed loss terms.

Babni et al. [4] generalized the trial construction $\tilde{u}(x, \theta) = P(x) + Q(x)N(x, \theta)$ by replacing $Q(x)$ with a flexible $\varphi(x)$, proving the error bound $|N_\varphi(t, \theta) - u(t)| \leq e^{\kappa|t-t_0|} L_\varphi(\theta)$. This confirms that minimizing the loss directly reduces solution error.

Audu et al. [6] compared PINN and ANN models on six first-order ODE tests. Their PINN achieved errors of 10^{-8} to 10^{-10} , significantly outperforming the ANN. This raises the question: can architecture optimization close this gap?

Pure ANN Approaches

Pratama et al. [2] found NeuroDiffEq’s Trial Approximate Solution trains without physics loss terms, aligning with our methodology, but only benchmarked on the heat equation which is linear. Dubey et al. [5] confirmed plain ANNs solve coupled ODE systems (errors $\sim 10^{-3}$), but used a fixed architecture. Okereke et al. [8] eliminated backpropagation via regression-based weight computation over polynomial fitting. Audu et al. [9] achieved 10^{-3} to 10^{-4} errors with a sigmoid ANN on three first-order ODEs. None of these studies explore how architecture choices affect performance.

Architecture and Loss Function Studies

Mall and Chakraverty [7] compared 4, 5, and 6 hidden nodes with random versus regression-based weights. Regression-based weights consistently outperform random ones, and accuracy plateaus beyond 6 nodes. However, they used only sigmoid activation on simple ODEs.

Lau and Werth [3] found that sigmoid/tanh require less training than ReLU/ELU, Xavier initialization accelerates convergence, and Adam outperforms SGD. They did not systematically vary depth and width.

Xiong [11] designed seven loss function constructions (polynomial, exponential, hyperbolic, logarithmic, logistic, sigmoid, softplus). Performance varies by problem: exponential works best for Newton’s cooling law, logarithmic for the motor suspension system.

Lorenz System Studies

Aslam et al. [12] solved Lorenz equations using ANN + PSO-NNA hybrid optimization (10 neurons, sigmoid, 90 parameters, 100 independent runs). However, they target a different Lorenz parameterization (σ, R, B) , rely on meta-heuristic optimization, and do not compare architectures.

Neural ODEs

Chen et al. [13] introduced Neural ODEs, parameterizing the derivative function $\frac{dh(t)}{dt} = f(h(t), t, \theta)$ with a neural network and using the adjoint method for backpropagation. This paradigm differs fundamentally from ours: Neural ODEs learn the dynamics (f), while we approximate the solution ($y(t)$) directly.

Hybrid Approaches

Muruges et al. [10] proposed a Neural-ODE Hybrid Block Method with the update rule:

$$y(t_{k+1}) = y(t_k) + h \sum_{j=0}^{m-1} \omega_j f_\theta(t_{k-j}, y_{k-j}) + \sum_{i=0}^{m-1} \alpha_i F(t_{k-i}, y_{k-i}) \quad (2.9)$$

where ω_j and α_j balance neural versus numerical contributions. It achieved machine-precision errors on harmonic oscillators and the Van der Pol equation.

2.3 Gap Analysis

Table 2.1 summarizes what each reviewed paper covers and where gaps exist.

Table 2.1: Literature Gap Analysis

Paper	Method	ODE/PDE	Arch. Study	Lorenz-1960	Plain ANN
PinnDE [1]	PINN+DeepONet	Both	No	Yes	No
ANN Survey [2]	PyDEns...	PDE	No	No	Yes
Neural ODEs [13]	Neural ODE	ODE	No	No	Partial
Aslam et al. [12]	ANN+PSO-NNA	ODE	No	Lorenz	No
ODEN [3]	Plain ANN	ODE	Partial	No	Yes
Mall & Chakraverty [7]	ANN	ODE	Yes	No	Yes
Xiong [11]	ANN	ODE	No	No	Yes
Babni et al. [4]	PINN	ODE	No	No	No
Dubey et al. [5]	Plain ANN	ODE	No	No	Yes
Okereke et al. [8]	ANN	ODE	No	No	Yes
Audu et al. (2024) [9]	Plain ANN	ODE	No	No	Yes
Audu et al. (2026) [6]	PINN vs ANN	ODE	No	No	Partial
Muruges et al. [10]	Hybrid	ODE	No	No	Partial
Our Work	Plain ANN	ODE	Yes	Yes	Yes

The literature review reveals five specific gaps that our research addresses:

1. **No architecture study for the Lorenz-1960 system:** Matthews and Bihlo [1] solve the Lorenz-1960 system but use a fixed DeepONet architecture without opti-

mization. Aslam et al. [12] study a different Lorenz parameterization. No existing paper systematically compares ANN architectures on the Lorenz-1960 system.

2. **No PINN vs. plain ANN comparison on Lorenz-1960:** Audu et al. [6] compared PINN and ANN on simple first-order ODEs. No study makes this comparison on a nonlinear, coupled ODE system like Lorenz-1960.
3. **No systematic activation function study for ODE solving:** Pratama et al. [2] use tanh throughout without ablation. Lau and Werth [3] note that sigmoid and tanh outperform ReLU but do not test modern alternatives like GELU or Swish. Mall and Chakraverty [7] use only sigmoid. No study compares activation functions (tanh, ReLU, sigmoid, GELU, Swish) on a coupled ODE system.
4. **No systematic depth and width study for ODE solution quality:** Mall and Chakraverty [7] test 4, 5, and 6 nodes on simple ODEs. Lau and Werth [3] observe that architecture should match equation complexity. But no paper systematically varies both depth (2–6 layers) and width (20–100 neurons) to find the optimal configuration for a Lorenz-type system.
5. **Limited generalization studies:** Most papers train for a single initial condition. The ability to generalize across different initial conditions remains largely unexplored for coupled nonlinear ODE systems.

Our research fills these gaps by performing a systematic architecture comparison on the Lorenz-1960 ODE system using plain ANNs. We vary depth, width, and activation functions while benchmarking against RK4 reference solutions and the PINN-based results from Matthews and Bihlo [1].

2.4 Summary

This chapter established the necessary background for our research. We reviewed the mathematical foundations of ODEs, the Lorenz-1960 system, classical numerical methods, and ANN-based solving approaches. The literature review covered 13 research papers spanning physics-informed methods, pure ANN approaches, architecture studies, Lorenz system solvers, Neural ODEs, and hybrid techniques.

The gap analysis reveals that no existing study performs a systematic architecture comparison of plain ANNs on the Lorenz-1960 ODE system. Existing works either use fixed architectures, focus on simpler equations, or rely on physics-informed loss terms. Our research directly addresses this gap by varying network depth, width, and activation functions to identify the optimal ANN architecture for solving the Lorenz-1960 system as an initial value problem.

Chapter 3

Project Design

[Must be present in FYDP-1 Report and also in Final Report]

Every chapter should start with 1-2 sentences on the outline of the chapter.

3.1 Requirement Analysis

3.1.1 Functional and Nonfunctional Requirements

3.1.2 Context Diagram

3.1.3 Data Flow Diagram Level 1

3.1.4 UI Design

3.2 Detailed Methodology and Design

You have to mention alternate solutions that you have considered. Why you have selected the specific solution, etc.

3.3 Project Plan

3.4 Task Allocation

3.5 Summary

Chapter 4

Implementation and Results

[Must be present in Final Report. Incomplete version might be included in FYDP-1 Report, however it is optional.]

Every chapter should start with 1-2 sentences on the outline of the chapter.

4.1 Environment Setup

4.2 Testing and Evaluation

4.3 Results and Discussion

4.4 Summary

Chapter 5

Standards and Design Constraints

[Must be present in FYDP-1 Report and also in Final Report]

Every chapter should start with 1-2 sentences on the outline of the chapter.

5.1 Compliance with the Standards

Only mention the standards that are related to your project. This list is not complete. For each of the standards discuss the alternates with pros and cons and rationale of selection.

5.1.1 Software Standards

5.1.2 Hardware Standards

5.1.3 Communication Standards

5.2 Design Constraints

Only mention the constraints that are related to the design of your project. This list is not complete.

5.2.1 Economic Constraint**5.2.2 Environmental Constraint****5.2.3 Ethical Constraint****5.2.4 Health and Safety Constraint****5.2.5 Social Constraint****5.2.6 Political Constraint****5.2.7 Sustainability****5.3 Cost Analysis**

Provide a cost analysis in terms of budget required and revenue model. In case of budget, you must show an alternate budget and rationales.

5.4 Complex Engineering Problem**5.4.1 Complex Problem Solving**

In this section, provide a mapping with problem solving categories. For each mapping add subsections to put rationale (Use Table 5.1). For P1, you need to put another mapping with Knowledge profile and rational thereof.

Table 5.1: Mapping with complex problem solving.

P1 Dept of Knowl- edge	P2 Range of Con- flicting Require- ments	P3 Depth of Analysis	P4 Familiarity of Issues	P5 Extent of Applicable Codes	P6 Extent of Stake- holder Involve- ment	P7 Inter- dependence
√	√					

5.4.2 Engineering Activities

In this section, provide a mapping with engineering activities. For each mapping add subsections to put rationale (Use Table 5.2).

5.5 Summary

Table 5.2: Mapping with complex engineering activities.

A1 Range of re- sources	A2 Level of Interac- tion	A3 Innovation	A4 Consequences for society and environment	A5 Familiarity
✓	✓			

Chapter 6

Conclusion

[Must be present in FYDP-1 Report and also in Final Report. Might be incomplete in FYDP-1 Report.]

Every chapter should start with 1-2 sentences on the outline of the chapter.

6.1 Summary

6.2 Limitation

6.3 Future Work

References

- [1] Jason Matthews and Alex Bihlo. Pinnde: physics-informed neural networks for solving differential equations. *arXiv preprint arXiv:2408.10011*, 2024.
- [2] Danang A Pratama, Maharani A Bakar, NB Ismail, and M Mashuri. Ann-based methods for solving partial differential equations: a survey. *Arab Journal of Basic and Applied Sciences*, 29(1):233–248, 2022.
- [3] Liam LH Lau and Denis Werth. Oden: A framework to solve ordinary differential equations using artificial neural networks. *arXiv preprint arXiv:2005.14090*, 2020.
- [4] Atmane Babni, Ismail Jamiai, and José Alberto Rodrigues. Error estimates and generalized trial constructions for solving odes using physics-informed neural networks. *Mathematical and Computational Applications*, 30(6):127, 2025.
- [5] Sourabh Kumar Dubey, Raghvendra Singh, and Hibah Islahi. Solving ordinary differential equations using artificial neural networks: A comprehensive approach. *Journal of Computational Analysis & Applications*, 33(6), 2024.
- [6] Khadeejah James Audu, Olalekan Abdrasheed Kayode, Fajuyi Samuel, Mubarak Inuolaji, and Yahaya Yusuph Amuda. Neural network solutions for first-order differential equations: A physics-informed perspective. *Journal of Engineering and Basic Sciences*, 5:1–13, 2026.
- [7] Susmita Mall and Snehashish Chakraverty. Comparison of artificial neural network architecture in solving ordinary differential equations. *Advances in Artificial Neural Systems*, 2013(1):181895, 2013.
- [8] Roseline N Okereke, Olaniyi S Maliki, and Ben I Oruh. A novel method for solving ordinary differential equations with artificial neural networks. *Applied Mathematics*, 12(10):900–918, 2021.
- [9] Khadeejah James Audu, Marshal Benjamin, Umar Mohammed, and Yusuph Amuda Yahaya. Utilizing the artificial neural network approach for the resolution of first-order ordinary differential equations. *Malaysian Journal of Science and Advanced Technology*, pages 210–216, 2024.

-
- [10] V Muruges, M Priyadharshini, Yogesh Kumar Sharma, Umesh Kumar Lilhore, Roobaea Alroobaea, Hamed Alsufyani, Abdullah M Baqasah, and Sarita Simaiya. A novel hybrid framework for efficient higher order ode solvers using neural networks and block methods. *Scientific Reports*, 15(1):8456, 2025.
 - [11] Xiao Xiong. New designed loss functions to solve ordinary differential equations with artificial neural network. *arXiv preprint arXiv:2301.00636*, 2022.
 - [12] Muhammad Naeem Aslam, Muhammad Waheed Aslam, Muhammad Sarmad Arshad, Zeeshan Afzal, Murad Khan Hassani, Ahmed M Zidan, and Ali Akgül. Neuro-computing solution for lorenz differential equations through artificial neural networks integrated with pso-nna hybrid meta-heuristic algorithms: a comparative study. *Scientific Reports*, 14(1):7518, 2024.
 - [13] Ricky TQ Chen, Yulia Rubanova, Jesse Bettencourt, and David K Duvenaud. Neural ordinary differential equations. *Advances in Neural Information Processing Systems*, 31, 2018.